

MINICURSO WEBDEV

PET SAÚDE

DIA 2



REPOSITÓRIO DO QITHUB

```
git clone https://github.com/davibital/minicurso-pet-saude
```

```
git checkout dia-2
```

```
npm install
```

```
npm run dev
```

RELEMBRANDO...

- O que é **React** e por que usar
- Componentes e JSX
- **Props** e **estado** (useState)
- React + TypeScript como base

TRABALHANDO COM FORMULÁRIOS

- Entrada de dados
- Registro de informação



TRABALHANDO COM FORMULÁRIOS

Principais tags **HTML**:

- **<form>**
- **<input>**
- **<label>**
- **<button type="submit">**

TRABALHANDO COM FORMULÁRIOS

O **<input/>** tem como propriedades:

- **type**: Define o tipo de entrada (numérica, texto, data);
- **id**: Identifica o input (serve como referência para a tag **<label/>**);
- **name**: Define o nome do campo ao submeter o formulário.

TRABALHANDO COM FORMULÁRIOS

O `<label/>` tem como propriedades:

- **for**: Referência direta para o id do input.

TRABALHANDO COM FORMULÁRIOS

```
1 <form id="formulario">
2   <label for="nome">Nome:</label><br />
3   <input type="text" id="nome" name="nome" value="Davi" /><br />
4   <label for="sobrenome">Sobrenome:</label><br />
5   <input
6     type="text"
7     id="sobrenome"
8     name="sobrenome"
9     value="Bittencourt"
10  /><br /><br />
11   <input type="submit" value="Submit" />
12 </form>
```

TRABALHANDO COM FORMULÁRIOS

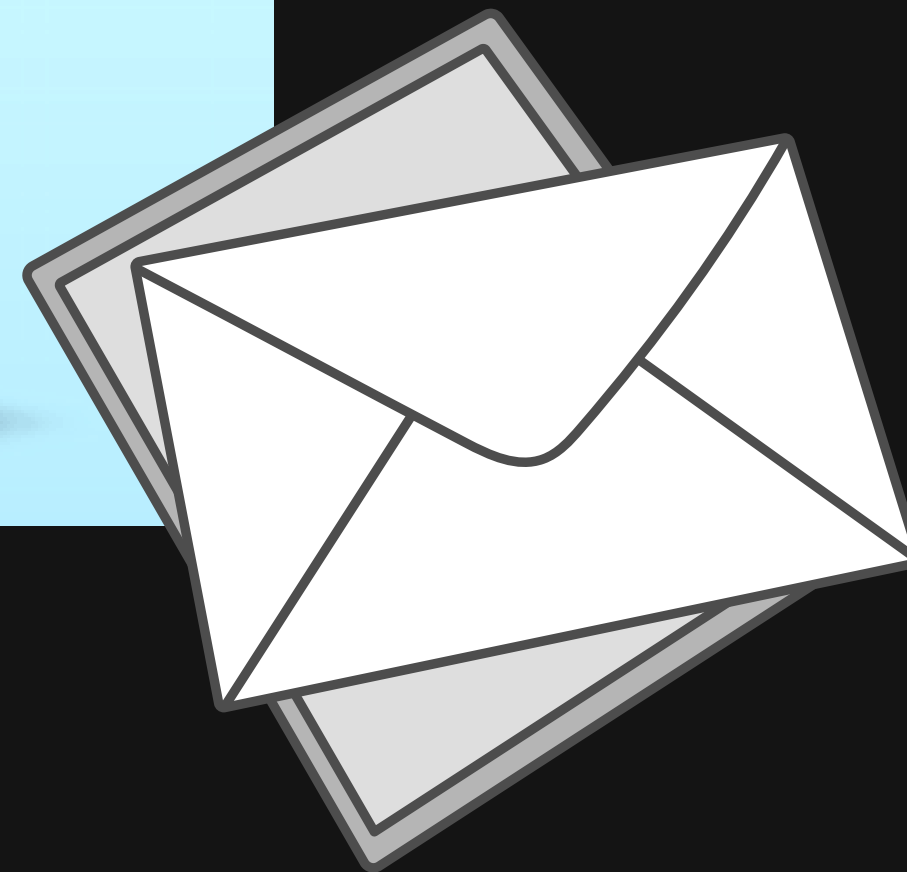
```
1 <form id="formulario">
2   <label for="nome">Nome:</label><br />
3   <input type="text" id="nome" name="nome" value="Davi" /><br />
4   <label for="sobrenome">Sobrenome:</label><br />
5   <input
6     type="text"
7     id="sobrenome"
8     name="sobrenome"
9     value="Bittencourt"
10  /><br /><br />
11  <input type="submit" value="Submit" />
12 </form>
```

```
1 function enviarDados(event) {
2   event.preventDefault();
3   const dados = new FormData(elementoFormulario);
4
5   alert(`{ nome: ${dados.get("nome")}, sobrenome: ${dados.get("sobrenome")} }`);
6 }
```

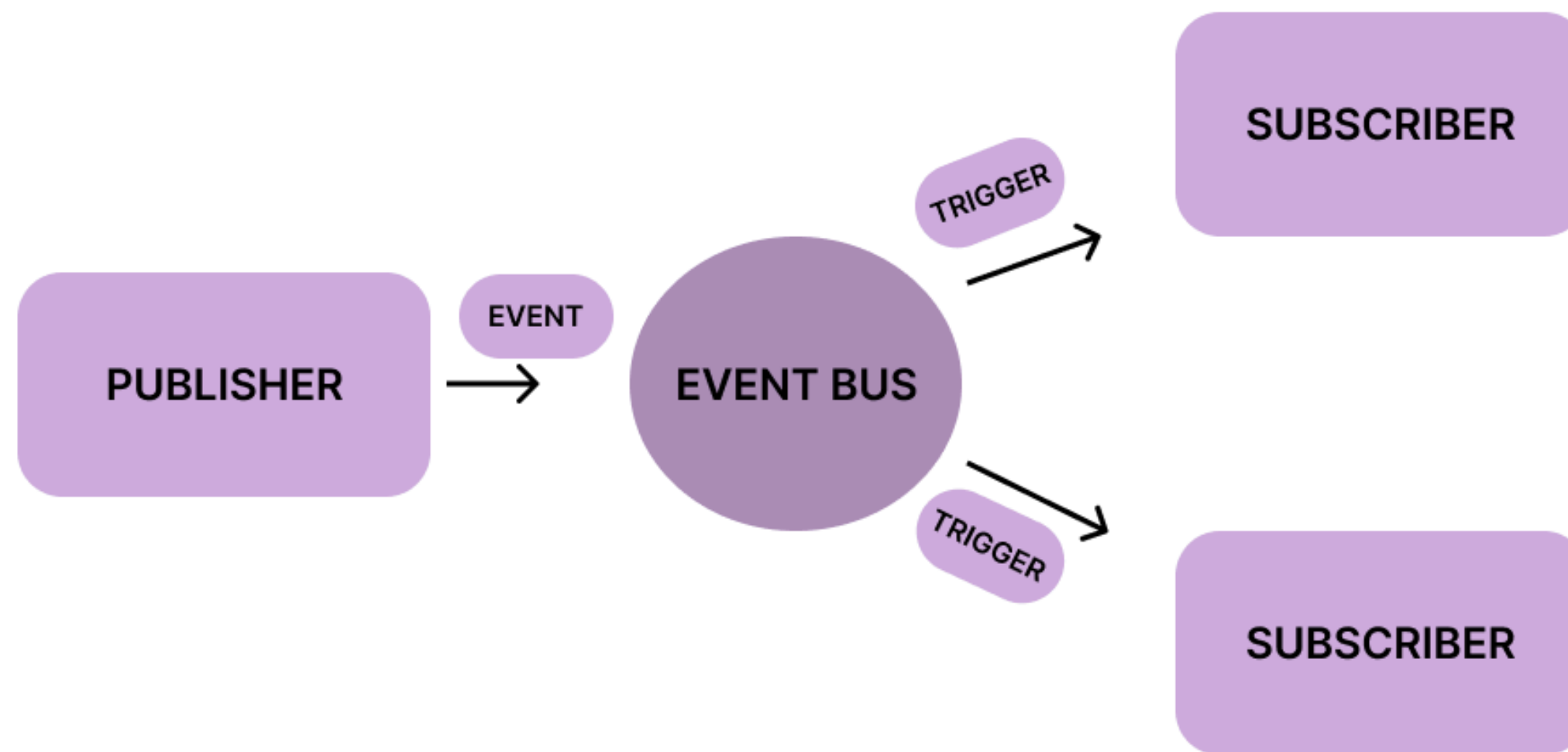

EVENTO **SUBMIT**

- Submit = submissão
- Propósito de **enviar dados**
- Botão com tipo **submit** emite esse evento

PUB - SUB



PUB - SUB



EVENTOS NO NAVEGADOR

Prefixo **on** + **nomeDoEvento**

Exemplos:

- **onClick** (ao clicar)
- **onInput** (ao inserir dados - digitar)
- **onScroll** (ao rolar o elemento)

LISTENERS NO NAVEGADOR

Reagem aos eventos.

Prefixo **handle** + **nomeDoEvento**

NO REACT



```
1 import { ChangeEvent, FormEvent, useState } from "react";
2 import { Medicamento, TipoUsoMedicamento } from "@types/medicamento";
3 import styles from "../page.module.css";
4 import { Spinner } from "@components/Spinner";
5 import { USO_MEDICAMENTO } from "@lib/constants";
6
7 const tiposUso: TipoUsoMedicamento[] = Object.values(
8   USO_MEDICAMENTO,
9 ) as TipoUsoMedicamento[];
```

NO REACT



```
1 export default function PaginaFormulario() {
2   const [nome, setNome] = useState<string>("");
3   const [intervalo, setIntervalo] = useState<number | null>(null);
4   const [dosagem, setDosagem] = useState<number | null>(null);
5   const [uso, setUso] = useState<TipoUsoMedicamento | null>(null);
6
7   const [erros, setErros] = useState<Record<string, string>>({});
8   const [carregando, setCarregando] = useState<boolean>(false);
```


NO REACT

```
1 // ...
2 <form onSubmit={handleSubmit} className={styles.form}>
3   <div className={styles.formGroup}>
4     <label htmlFor="nome" className={styles.label}>
5       Nome do Medicamento
6     </label>
7     <input
8       name="nome"
9       id="nome"
10      type="text"
11      value={nome ?? ""}
12      onChange={handleNomeChange}
13      className={styles.input}
14      placeholder="Ex: Dipirona"
15    />
16    {erros.nome && <p className={styles.error}>{erros.nome}</p>}
17  </div>
18  {/* resto do formulário */}
```


NO REACT



```
1  const handleNomeChange = (event: ChangeEvent<HTMLInputElement>) => {  
2    event.currentTarget.setCustomValidity("");  
3    setNome(event.target.value);  
4  };
```

NO REACT

```
1  const handleSubmit = (event: FormEvent<HTMLFormElement>) => {
2    event.preventDefault();
3    setCarregando(true);
4
5    if (!validarFormulario()) {
6      setCarregando(false);
7      return;
8    }
9
10   const medicamento: Medicamento = {
11     nome,
12     dosagem: `${dosagem}mg`,
13     intervalo: `${intervalo}h`,
14     uso: uso as TipoUsoMedicamento,
15   };
16
17   setTimeout(() => {
18     setCarregando(false);
19     alert("Medicamento salvo: " + JSON.stringify(medicamento));
20   }, 3000);
21 }
```

NO REACT

```
1  const validarFormulario = () => {
2    const novosErros: Record<string, string> = {};
3
4    if (!nome || nome.trim() === "") {
5      novosErros.nome = "É necessário informar o nome do medicamento.";
6    }
7
8    if (!intervalo || isNaN(intervalo)) {
9      novosErros.intervalo =
10        "É necessário informar o intervalo do medicamento.";
11    }
12
13    if (!dosagem || isNaN(dosagem)) {
14      novosErros.dosagem = "É necessário informar a dosagem do medicamento.";
15    }
16
17    if (!uso) {
18      novosErros.uso = "É necessário selecionar um tipo de uso de medicamento.";
19    }
20
21    setErros(novosErros);
22    return Object.keys(novosErros).length === 0;
23  };
24
```

BIBLIOTECAS DE FORMULÁRIOS

- **Menos** código repetitivo
- Melhor **controle** de estado
- Melhor **integração** com validação
- **Performance** em formulários grandes

BIBLIOTECAS DE FORMULÁRIOS



React Hook Form



FORMIK

REACT HOOK FORM



```
1  import { useForm } from "react-hook-form";
2
3  type FormData = {
4    nome: string;
5    intervalo: number;
6    dosagem: number;
7    uso: TipoUsoMedicamento;
8  };
9
10 export default function PaginaFormulario() {
11   const [carregando, setCarregando] = useState<boolean>(false);
12
13   const {
14     register,
15     handleSubmit,
16     formState: { errors },
17     reset,
18   } = useForm<FormData>();
```

REACT HOOK FORM



```
1 // ...
2 <form onSubmit={handleSubmit(onSubmit)} className={styles.form}>
3   <div className={styles.formGroup}>
4     <label htmlFor="nome" className={styles.label}>
5       Nome do Medicamento
6     </label>
7     <input
8       {...register("nome", {
9         required: "É necessário informar o nome do medicamento.",
10        validate: (value) =>
11          value.trim() !== "" || "Nome não pode ser vazio.",
12        })}
13     id="nome"
14     type="text"
15     className={styles.input}
16     placeholder="Ex: Dipirona"
17   />
18   {errors.nome && (
19     <p className={styles.error}>{errors.nome.message}</p>
20   )}
21 </div>
22 {/* resto do formulário */}
```

REACT HOOK FORM



```
1  const onSubmit = (data: FormData) => {
2    setCarregando(true);
3
4    const medicamento: Medicamento = {
5      nome: data.nome,
6      dosagem: `${data.dosagem}mg`,
7      intervalo: `${data.intervalo}h`,
8      uso: data.uso,
9    };
10
11    setTimeout(() => {
12      setCarregando(false);
13      alert("Medicamento salvo: " + JSON.stringify(medicamento));
14      reset();
15    }, 3000);
16  };
```

FORMIK



```
1 import { Formik, Form, Field, ErrorMessage } from "formik";
2
3 type FormValues = {
4   nome: string;
5   intervalo: number | "";
6   dosagem: number | "";
7   uso: TipoUsoMedicamento | "";
8 };
9
10 export default function PaginaFormulario() {
11   const initialValues: FormValues = {
12     nome: "",
13     intervalo: "",
14     dosagem: "",
15     uso: "",
16   };
17 }
```


FORMIK



```
1 // ...
2 <Formik
3   initialValues={initialValues}
4   onSubmit={handleSubmit}
5 >
6   {({ isSubmitting }) =>
7     isSubmitting ? (
8       <Spinner />
9     ) : (
10      <Form className={styles.form}>
11        <div className={styles.formGroup}>
12          <label htmlFor="nome" className={styles.label}>
13            Nome do Medicamento
14          </label>
15          <Field
16            name="nome"
17            id="nome"
18            type="text"
19            className={styles.input}
20            placeholder="Ex: Dipirona"
21          />
22          <ErrorMessage
23            name="nome"
24            component="p"
25            className={styles.error}
26          />
27        </div>
28      {/* resto do formulário */}
```


FORMIK



```
1  const handleSubmit = (  
2    values: FormValues,  
3    {  
4      resetForm,  
5      setSubmitting,  
6    }: {  
7      resetForm: () => void;  
8      setSubmitting: (isSubmitting: boolean) => void;  
9    },  
10  ) => {  
11    const medicamento: Medicamento = {  
12      nome: values.nome,  
13      dosagem: `${values.dosagem}mg`,  
14      intervalo: `${values.intervalo}h`,  
15      uso: values.uso as TipoUsoMedicamento,  
16    };  
17  
18    setTimeout(() => {  
19      alert("Medicamento salvo: " + JSON.stringify(medicamento));  
20      setSubmitting(false);  
21      resetForm();  
22    }, 3000);  
23  };
```

VALIDAÇÃO

Como validar as entradas de forma mais **simples** e **robusta**?

Vamos ver algumas soluções...

VALIDAÇÃO



Zod

Yup

ZOD

```
1 import { z } from "zod";
2 import { USO_MEDICAMENTO } from "@/lib/constants";
3 import { TipoUsoMedicamento } from "@/types/medicamento";
4
5 const tiposUso: TipoUsoMedicamento[] = Object.values(
6   USO_MEDICAMENTO,
7 ) as TipoUsoMedicamento[];
8
9 // Schema para formulário (antes de transformar os dados)
10 export const medicamentoFormSchema = z.object({
11   nome: z
12     .string()
13     .min(1, "É necessário informar o nome do medicamento.")
14     .trim()
15     .refine((val) => val.length > 0, {
16       message: "Nome não pode ser vazio.",
17     }),
18   intervalo: z
19     .number("É necessário informar o intervalo do medicamento.")
20     .positive("Intervalo deve ser um número positivo."),
21   dosagem: z
22     .number("É necessário informar a dosagem do medicamento.")
23     .positive("Dosagem deve ser um número positivo."),
24   uso: z.enum(tiposUso as TipoUsoMedicamento[], {
25     message: "É necessário selecionar um tipo de uso de medicamento.",
26   }),
27 });
28
29 // Types inferidos (prefixo com o nome do schema)
30 export type MedicamentoFormData = z.infer<typeof medicamentoFormSchema>;
```


ZOD

```
1 import { useForm } from "react-hook-form";
2 import { zodResolver } from "@hookform/resolvers/zod";
3
4 export default function PaginaFormulario() {
5   const [carregando, setCarregando] = useState<boolean>(false);
6
7   const {
8     register,
9     handleSubmit,
10    formState: { errors },
11    reset,
12  } = useForm<MedicamentoFormData>({
13    resolver: zodResolver(medicamentoFormSchema),
14  });
```


ZOD



```
1  <input
2    {...register("nome")}
3    id="nome"
4    type="text"
5    className={styles.input}
6    placeholder="Ex: Dipirona"
7  />
8  {errors.nome && (
9    <p className={styles.error}>{errors.nome.message}</p>
10 )}
11
```

YUP

```
1  const validationSchema = Yup.object({
2    nome: Yup.string()
3      .required("É necessário informar o nome do medicamento.")
4      .trim()
5      .min(1, "Nome não pode ser vazio."),
6    intervalo: Yup.number()
7      .required("É necessário informar o intervalo do medicamento.")
8      .typeError("Intervalo deve ser um número válido."),
9    dosagem: Yup.number()
10     .required("É necessário informar a dosagem do medicamento.")
11     .typeError("Dosagem deve ser um número válido."),
12    uso: Yup.string()
13     .required("É necessário selecionar um tipo de uso de medicamento.")
14     .oneOf(tiposUso, "Tipo de uso inválido."),
15  });
```

YUP



```
1  <Formik
2    initialValues={initialValues}
3    validationSchema={validationSchema}
4    onSubmit={handleSubmit}
5  >
6    {/* resto do formulário */}
```

ZOD E YUP

Acrescenta **robustês** ao código

Contribui com a **escalabilidade**

Zod tem compatibilidade maior com o **React Hook Form**

Yup tem compatibilidade maior com o **Formik**

VAMOS PRATICAR!

Vamos criar um formulário para cadastrar equipamentos de um hospital.

VAMOS PRATICAR!

```
1 export type Equipamento = {
2   id: string;
3   nome: string;
4   categoria: CategoriaEquipamento;
5   status: StatusEquipamento;
6   prioridade: PrioridadeEquipamento;
7   localizacao: string;
8
9   // Data em formato ISO 8601
10  dataAquisicao: string;
11  dataUltimaManutencao: string;
12
13  // Data em formato ISO 8601
14  criadoEm: string;
15  atualizadoEm: string;
16 };
```

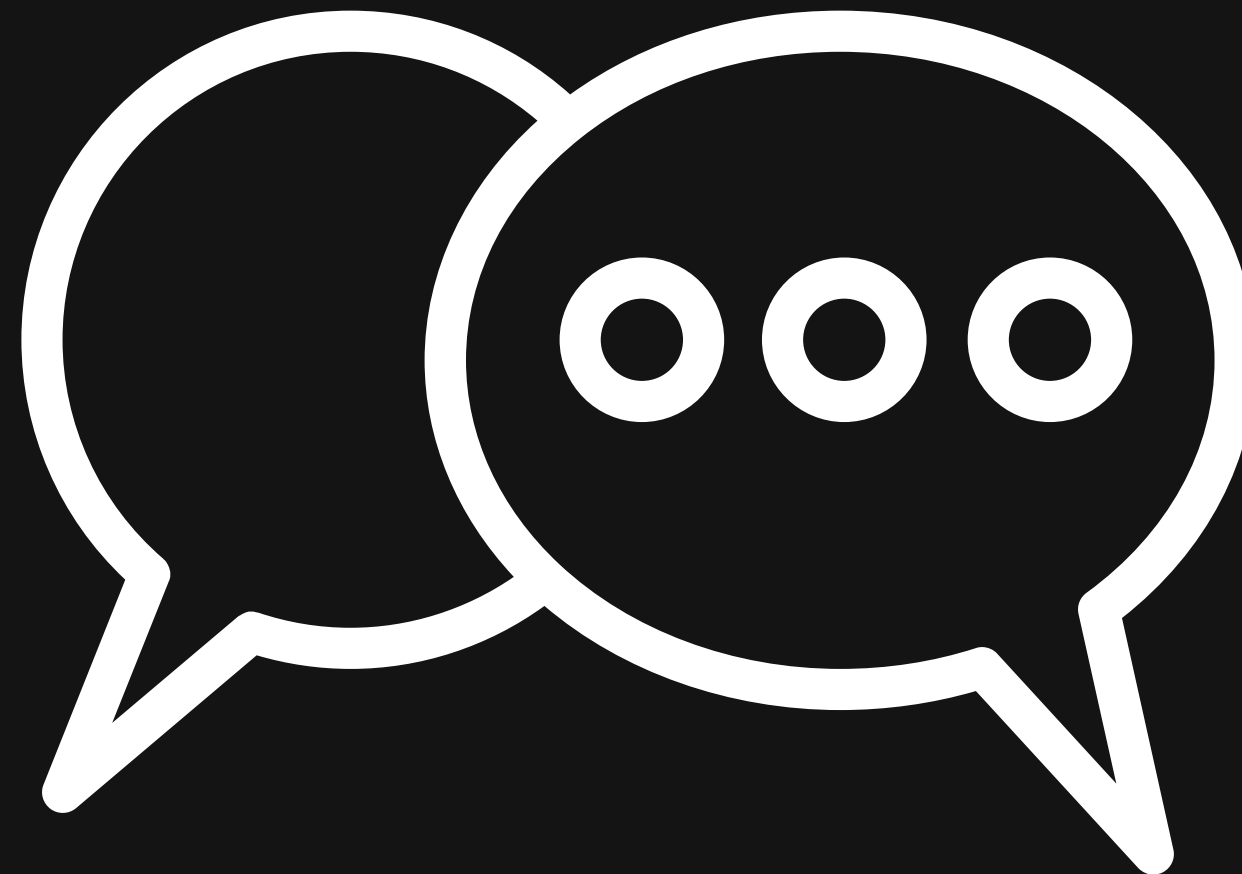
VAMOS PRATICAR!

```
1 export const USO_MEDICAMENTO = {
2   ORAL: "Oral",
3   INTRAMUSCULAR: "Intramuscular",
4   CUTANEA: "Cutânea",
5   NASAL: "Nasal",
6 } as const;
7
8 export const PRIORIDADE_EQUIPAMENTO = {
9   BAIXA: "Baixa",
10  MEDIA: "Média",
11  ALTA: "Alta",
12 } as const;
13
14 export const STATUS_EQUIPAMENTO = {
15  DISPONIVEL: "Disponível",
16  EM_USO: "Em Uso",
17  MANUTENCAO: "Manutenção",
18  INATIVO: "Inativo",
19  DESCARTADO: "Descartado",
20 } as const;
21
22 export const CATEGORIA_EQUIPAMENTO = {
23  MONITOR: "Monitor",
24  RESPIRADOR: "Respirador",
25  BOMBA_INFUSAO: "Bomba de Infusão",
26  RAO_X: "Raio-X",
27  ULTRASSOM: "Ultrassom",
28  MACA: "Maca",
29  OUTRO: "Outro",
30 } as const;
```

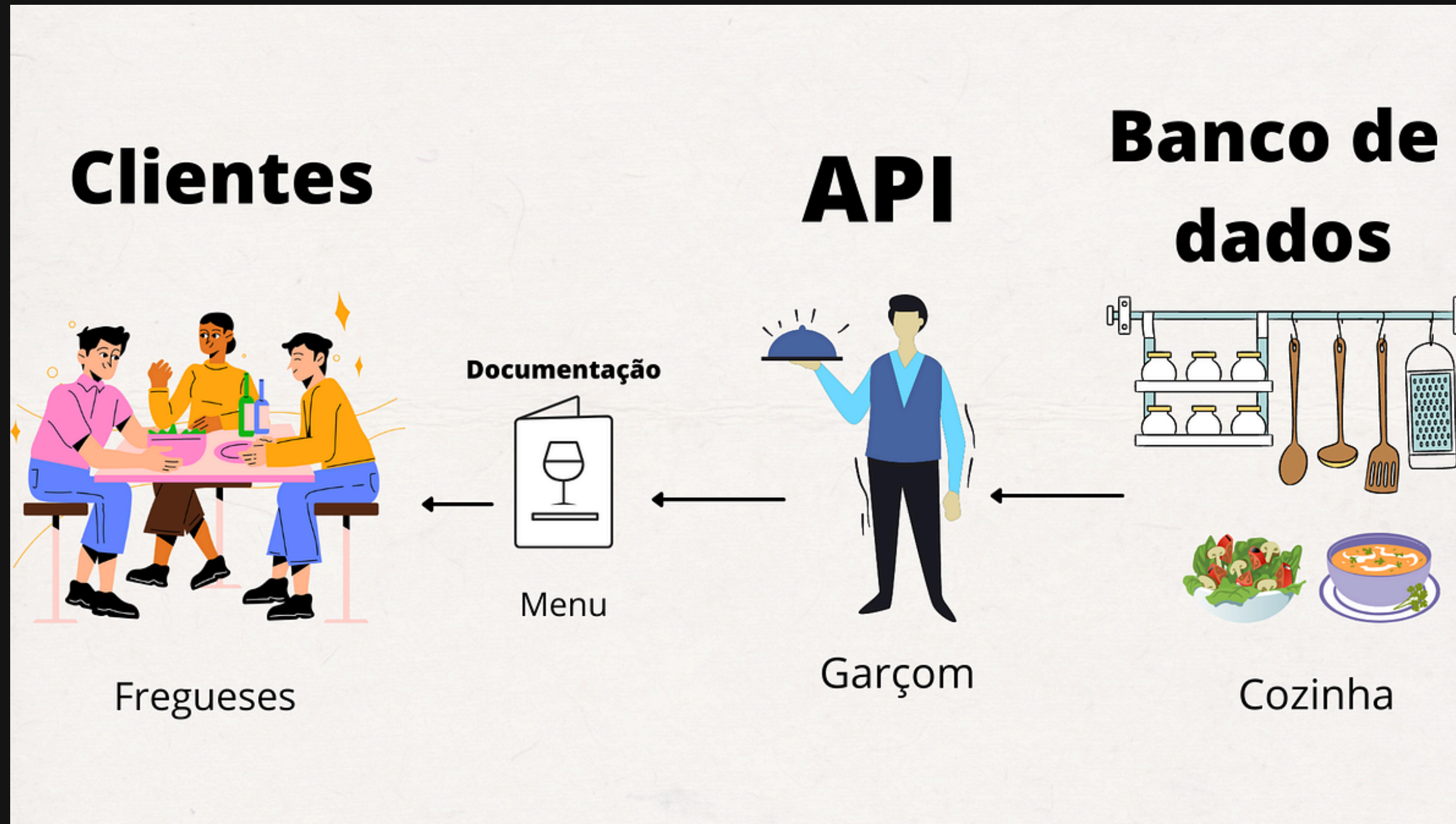
API

(Application Programming Interface)

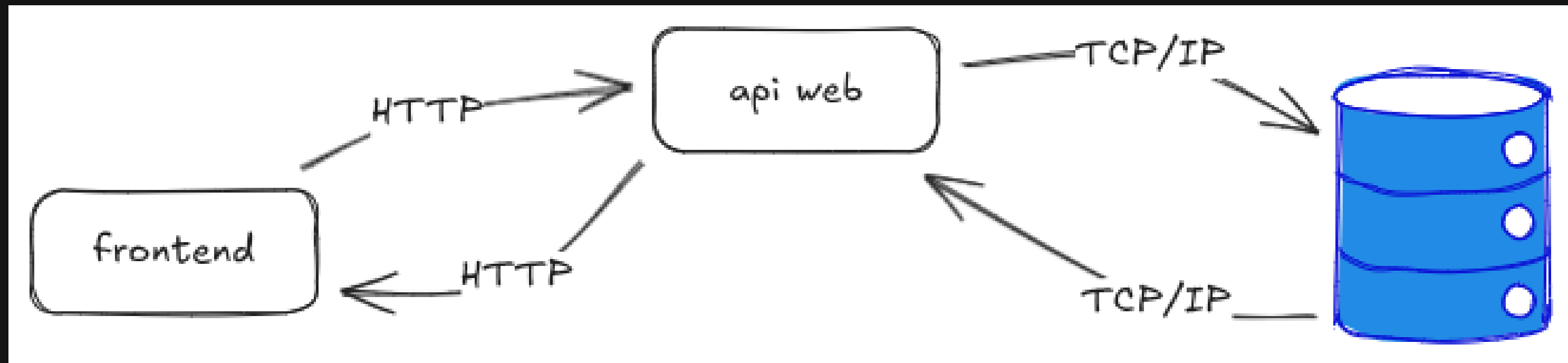
Conjunto de regras, protocolos e ferramentas que permite a **comunicação** entre diferentes softwares e componentes.



API WEB



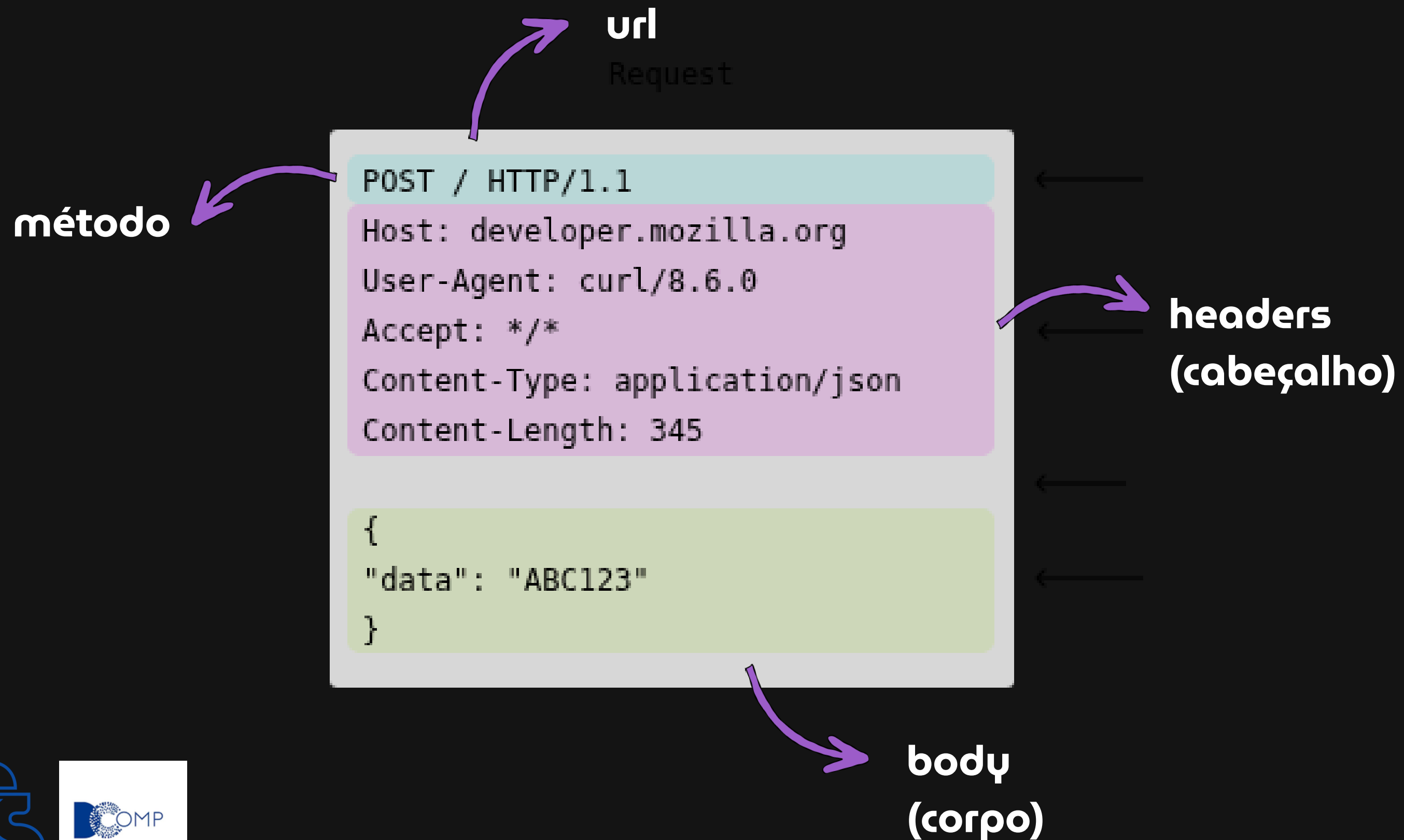
API WEB



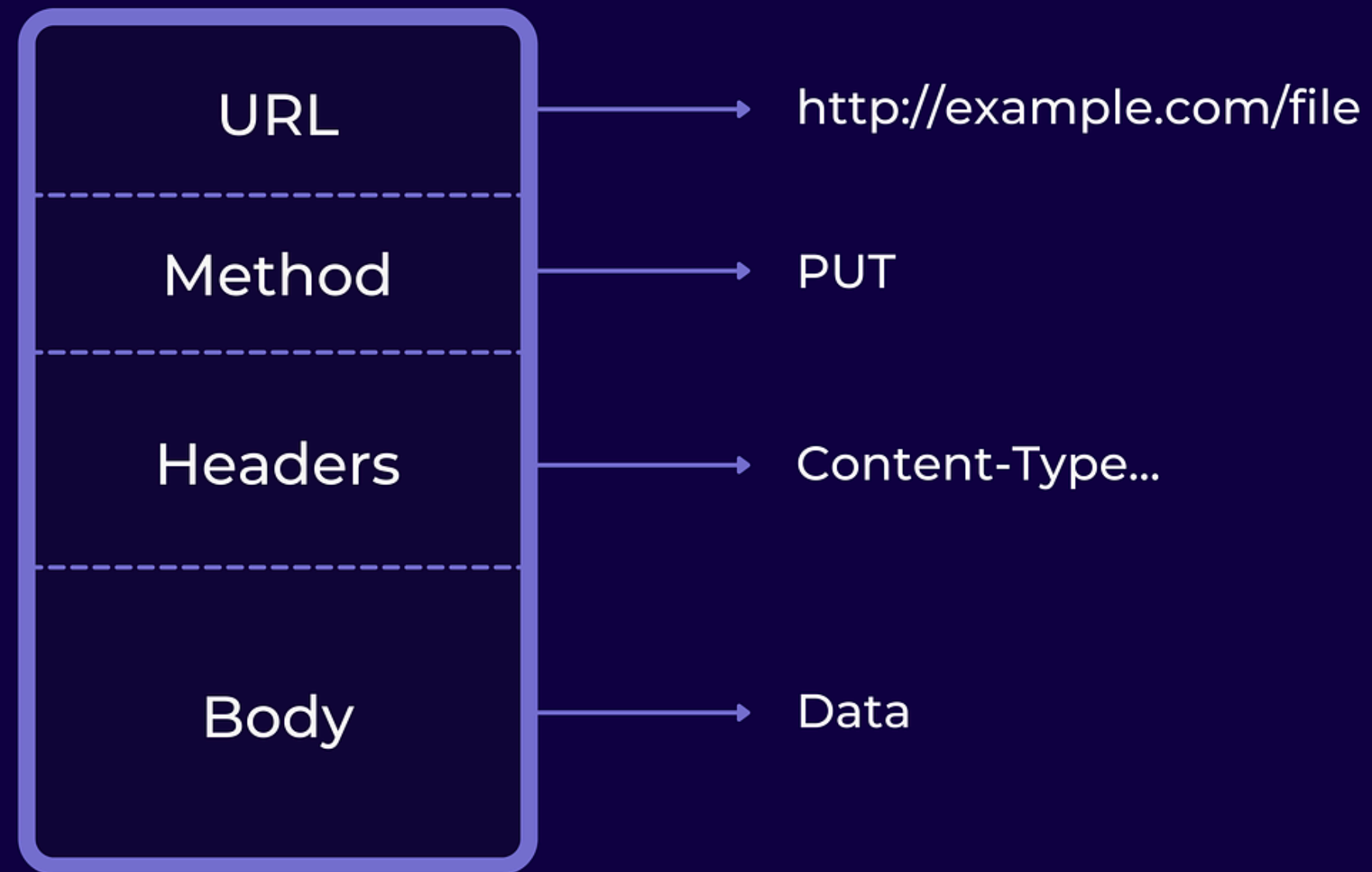
MÉTODOS HTTP

- **GET**: buscar dados
- **POST**: criar dados
- **PUT/PATCH**: atualizar dados
- **DELETE**: excluir dados

ESTRUTURA REQUISIÇÃO



ESTRUTURA REQUISIÇÃO



FETCH

API **nativa** do JavaScript para realizar **requisições HTTP**.

Sempre irá retornar uma **Promise**.



PROMISES

As **Promises** representam operações assíncronas.

Podem ser tratadas de duas formas:

- **then / catch**
- **async / await**

PROMISES

then / catch



```
1 // Exemplo de consumo da API do Rick and Morty
2   fetch("https://rickandmortyapi.com/api/character/1")
3     .then((res) => res.json())
4     .then((data) => setCharacter(data));
```

async / await



```
1 const res = await fetch("https://rickandmortyapi.com/api/character/1");
2 const data = await res.json();
3 setCharacter(data);
```

PROMISES

Definindo requisição completa com **método**, **cabeçalho** e **corpo**.

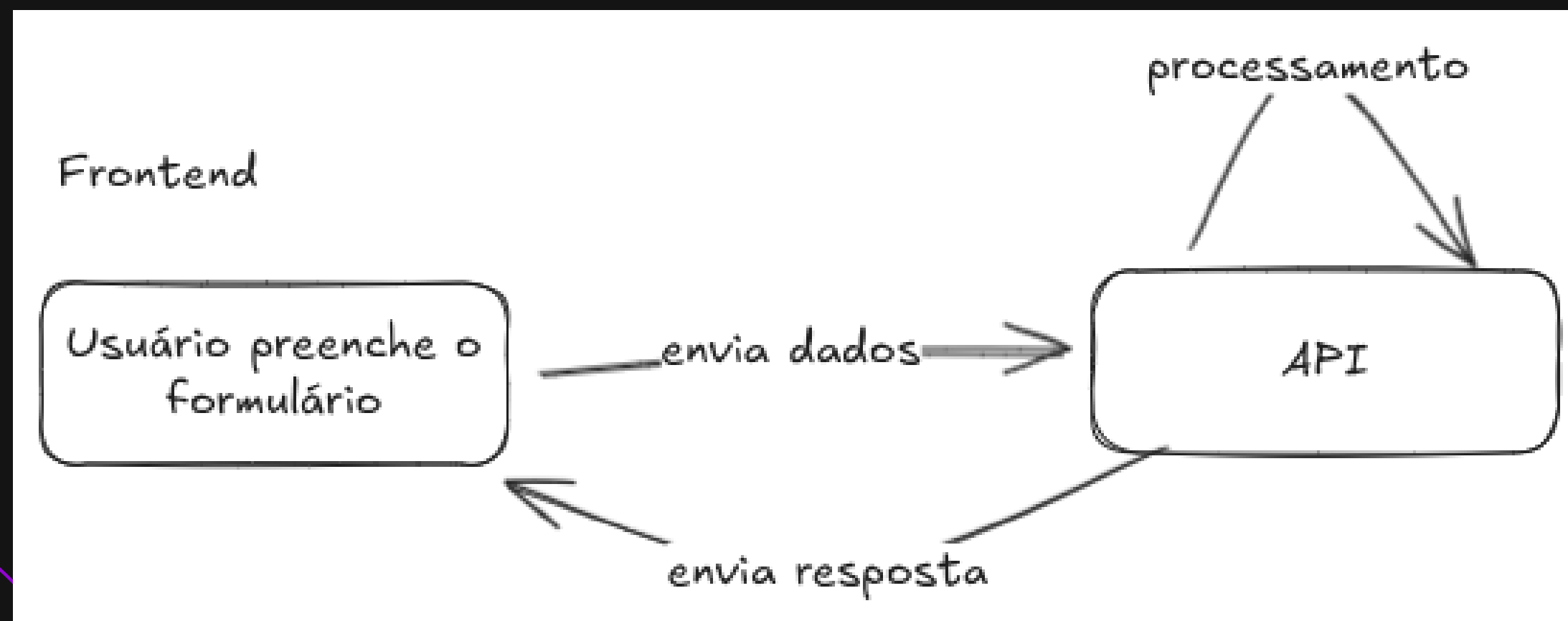
```
1  fetch("https://url-da-api.com/api/usuarios", {
2    method: "POST",
3    headers: {
4      "Content-Type": "application/json",
5    },
6    body: JSON.stringify({
7      nome: "Davi Bittencourt",
8      email: "davi@email.com",
9      senha: "coxinha123",
10    }),
11  });
```

VAMOS PRATICAR!

Vamos **integrar** o nosso formulário à uma **API** interna local.

Documentação no arquivo **</pet-saude/docs/API.md>**

VAMOS PRATICAR!



BIBLIOTECAS CSS

Aumentam a produtividade com **estilos prontos**.

Adicionam uma maior **consistência visual**.

BOOTSTRAP

Baseado em componentes.

Focado em responsividade.

Menos flexível.



BOOTSTRAP



```
1 <div class="card">
2   
3   <div class="card-body">
4     <h5 class="card-title">Título do Cartão</h5>
5     <p class="card-text">Um texto qualquer escrito dentro do cartão, servindo como uma legenda ou até mesmo um complemento de informações.</p>
6     <a href="#" class="btn btn-primary">Ir até →</a>
7   </div>
8 </div>
```

300 x 200

Powered by HTML.COM

Título do Cartão

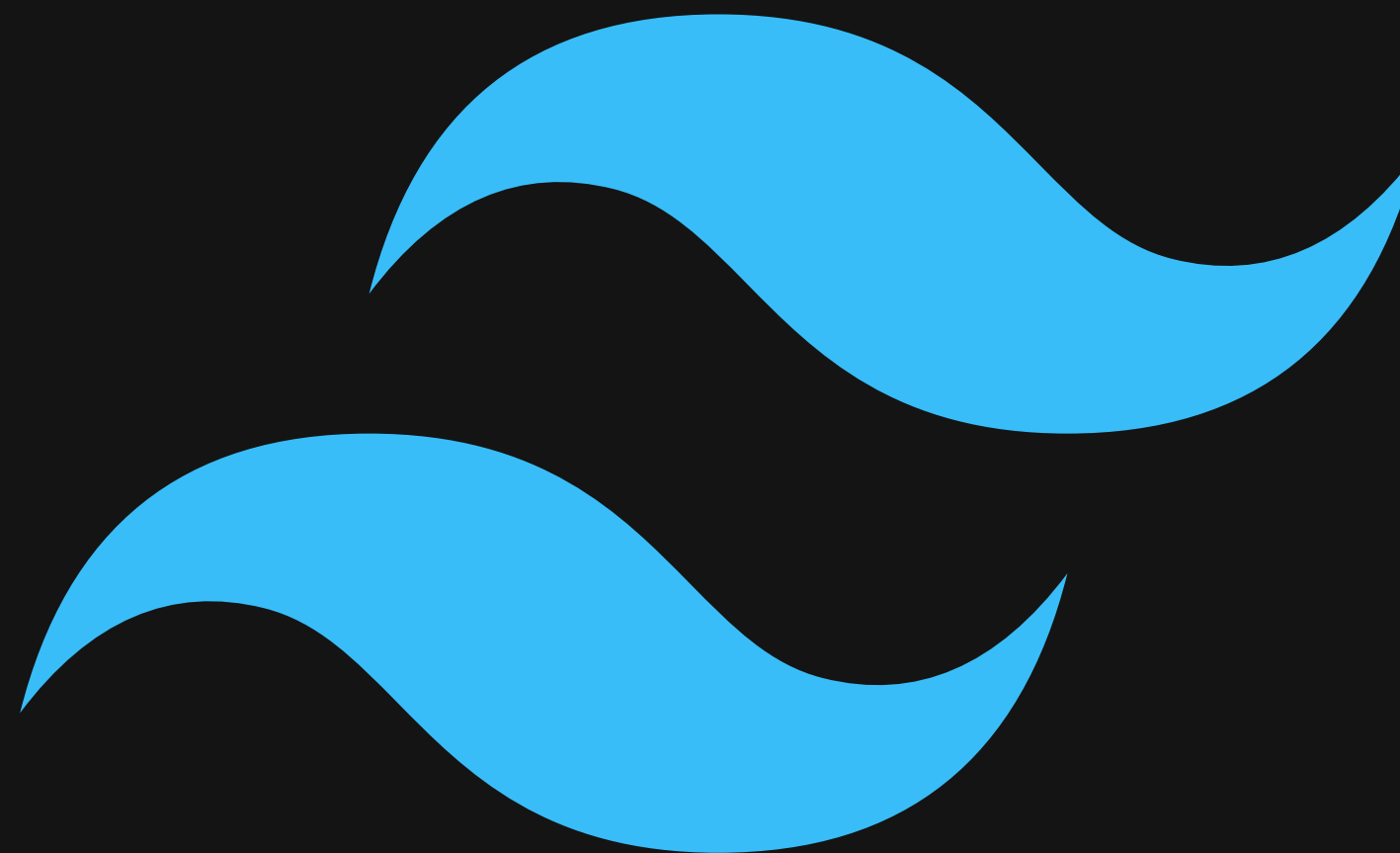
Um texto qualquer escrito dentro do cartão, servindo como uma legenda ou até mesmo um complemento de informações.

Ir até ->

TAILWIND CSS

Classes utilitárias.

Maior flexibilidade.



TAILWIND CSS

```
1 <div class="flex flex-col items-center gap-6 p-7 md:flex-row md:gap-8 rounded-2xl">
2   <div>
3     
4   </div>
5   <div class="flex items-center md:items-start">
6     <span class="text-2xl font-medium">Class Warfare</span>
7     <span class="font-medium text-sky-500">The Anti-Patterns</span>
8     <span class="flex gap-2 font-medium text-gray-600 dark:text-gray-400">
9       <span>No. 4</span>
10      <span>·</span>
11      <span>2025</span>
12    </span>
13  </div>
14 </div>;
```

TAILWIND CSS



Class Warfare

The Anti-Patterns

No. 4 · 2025

BIBLIOTECAS DE COMPONENTES

Fornece toda a **lógica visual** e **estilização** para componentes comuns.

Bibliotecas mais populares:

- **Shadcn/ui** - baseada em TailwindCSS
- **MUI Material** - componentes da Google

SHADCN/UI

Baixa o código do componente direto no projeto (ele pode ser modificado).

Dependência direta do **Tailwind CSS**.

Construída em cima do **Radix UI**.

MUI MATERIAL

Estilização baseada em **CSS puro**.

Fornece componentes de **layout**.

QRÁFICOS

Melhor visualização de dados.

Facilita a tomada de decisão.

Lógica de renderização encapsulada em componentes.

GRÁFICOS

Bibliotecas populares:

- **Recharts**
- **react-chartjs-2**
- **@mui/x-charts**

VAMOS PRATICAR!

Vamos nos basear no que construímos até agora para tentar criar uma aplicação completa de gerenciamento de equipamentos.

Vamos exercitar a **manipulação de formulários**, **consumo de APIs** e visualização de dados em **gráficos**.

PRÁTICA

Casos de uso:

- Cadastrar Equipamentos
- Listar Equipamentos (Com **filtros**)
- Alterar **status** do Equipamento
- Visualizar situação dos Equipamentos em **gráficos**

OBRIQADO!



<https://www.instagram.com/lawd.ufs>

<https://www.linkedin.com/company/lawd-ufs/>